ACTIVIDAD 3.3 / 01

Sistema de observabilidad end-to-end

Laboratorio integrador con AIOps, observabilidad de red y resiliencia

Maestría en Arquitectura de Software
Observabilidad en Ambientes Productivos
Universidad de La Sabana
18 de octubre de 2026

Resumen ejecutivo. Se construyó y ejecutó un sandbox aislado con tres microservicios FastAPI, una base PostgreSQL, instrumentación OpenTelemetry de métricas, logs y trazas, proxies Envoy y detección automática correlacionada. Se conservaron los registros crudos para auditar cada resultado.

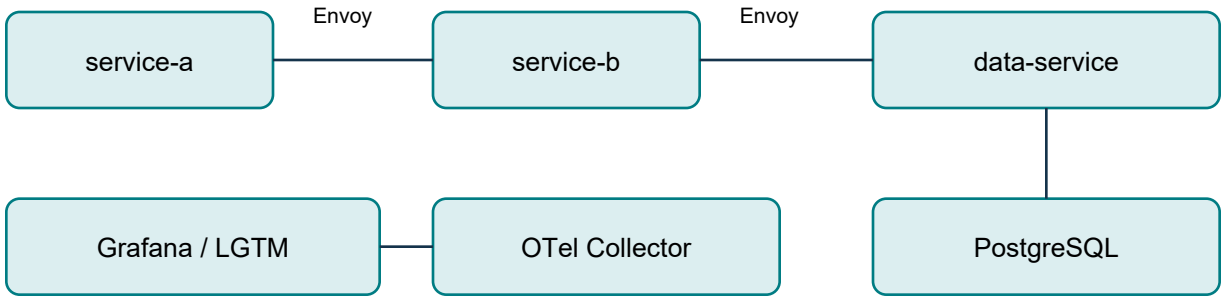
Resultado medido	Valor
Alerta por 200 ms en service-b	8.83 s
Alerta por errores en data-service	7.43 s
Comprobaciones de evidencias	11/11 satisfactorias
Entorno efectivamente ejecutado	Docker local; cloud.provider=local
Despliegue en AWS y GCP	Pendiente; sesión AWS disponible en us-east-2

Conclusión de alcance. La entrega demuestra el núcleo local y los dos experimentos, pero no acredita el 100% de los requisitos. Cloud SQL, RDS, service mesh gestionado, AIOps nativo, VPC Flow Logs y SCC/Security Hub requieren despliegue y evidencia cloud. La infraestructura adjunta fue validada, no aplicada. No se solicita asignar una nota automática.

La modalidad del enunciado es grupal de tres integrantes. No se recibieron los nombres del equipo; deben incorporarse en la sustentación. El repositorio y los artefactos permiten reproducir el trabajo sin atribuir intervenciones personales no verificadas.

Arquitectura y límites del sistema

La petición externa entra por service-a, continúa por Envoy hacia service-b y luego por un segundo Envoy hacia data-service. Este último consulta una tabla real de PostgreSQL. El contexto W3C viaja por HTTP; la aplicación propaga el trace_id en la respuesta para facilitar la investigación.



Capa	Implementación y contrato
Aplicación	Tres contenedores del mismo código, rol por SERVICE_NAME; /work y /health.
Datos	PostgreSQL 16; tabla inventory con PK y restricción quantity >= 0.
Telemetría	SDK OTel -> OTLP HTTP -> Collector -> Prometheus, Tempo y Loki.
Red	Dos proxies Envoy con métricas L7 y access logs; red Docker dedicada.
Operación	Detector continuo, reglas Prometheus, Grafana y controles de fallos con TTL.

Aislamiento. Todos los puertos publicados se limitan a 127.0.0.1. La base no tiene puerto público. Los servicios de aplicación usan usuario sin privilegios, filesystem de solo lectura, capacidades retiradas y límites de CPU/memoria. Las credenciales aleatorias se generan en .env ignorado por Git.

Frontera cloud. El diseño de destino mantiene la cadena y sustituye PostgreSQL local por Cloud SQL o RDS privado. No se representa un segundo proveedor mediante una simple etiqueta local. Envoy local no demuestra mTLS, un control plane ni una malla gestionada.

Tres pilares y correlación verificable

La instrumentación automática de FastAPI y HTTPX produce spans de servidor y cliente; psycopg2 instrumenta el acceso real a la base. El span SELECT inventory agrega convenciones actuales de base de datos sin registrar contraseñas ni valores de negocio. Los atributos db.system.name, db.namespace y db.operation.name permiten filtrar consultas.

Pilar	Prueba conservada	Uso operacional
Trazas	trace-success.json y trace-failed.json	Camino A/B/data; error y consulta SQL.
Métricas	metrics.json, targets.json	Solicitudes, códigos HTTP, duración por servicio.
Logs	logs.json y logs del Collector	Eventos estructurados con service y trace_id.
Red L7	network-metrics.json y mesh-*.log	Ruta, upstream, código, tiempo y traceparent.

Cadena de evidencia. verify.py comprueba que una misma traza exitosa contenga los tres servicios y PostgreSQL, que Loki haya persistido logs correlacionables y que existan métricas de los tres servicios. No basta con mostrar el SDK instalado o un panel vacío.

Precisión del p99. El histograma usa límites explícitos de 5, 10, 25, 50, 100, 150, 200, 250, 300, 500, 1000, 2000 y 5000 ms. Esto evita que buckets genéricos produzcan una interpolación inútil para un SLO de 200 ms. El p99 del cliente se obtiene de las solicitudes CSV; el de Prometheus sigue siendo una aproximación por buckets.

DataOps. data-quality.txt registra cantidad de filas y filas inválidas sobre la base real. El esquema se crea de manera reproducible con init.sql y no se modifica durante caos. No existe todavía monitorización de frescura, linaje ni calidad histórica; se incluye en el roadmap.

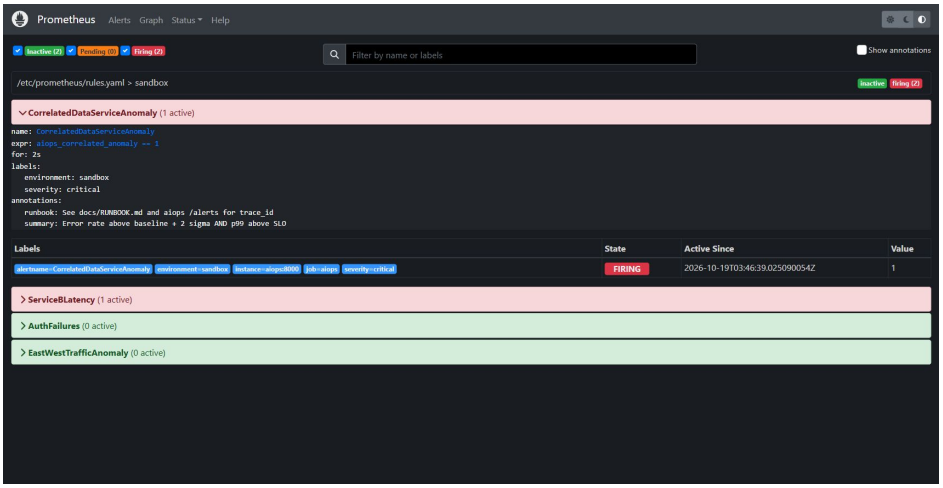
Retención local: traces 24 h; datos del ensayo quedan exportados en Git. La retención cloud, el control de cardinalidad y el muestreo de producción necesitan decisiones adicionales. Se evitan trace_id como etiquetas Prometheus para no multiplicar series.

AIOps: baseline, correlación y ruido

El detector aprende 12 ventanas iniciales de 30 probes cada una. Congela la media (0.0000) y la desviación poblacional (0.0000) de la tasa de errores; el umbral resultante es 0.0000. La espera de arranque separa la disponibilidad inicial del baseline operacional.

Regla exigida. error_rate > media_baseline + 2 × sigma **Y** p99 > 0.2 segundos. La alerta enriquecida incluye data-service, tasa, p99, umbral, trace_id fallido, acción sugerida y runbook. Prometheus evalúa aiops_correlated_anomaly cada 2 s y requiere 2 s de permanencia.

Control de ruido medido	Resultado
Ventanas independientes evaluadas	12
Ventanas con alerta estática (algún 5xx)	12
Ventanas con ambas señales	0
Reducción de ventanas alertadas	100.0%



La comparación aplica a la carga de errores rápidos de este ensayo; no es una estimación general de falsos positivos. No se dispone de un conjunto etiquetado para medir precisión/recall. Una regla AND suprime errores sin latencia y también puede ocultar incidentes reales de ese tipo; se recomienda mantener una alerta separada por disponibilidad.

Esta es detección estadística local, no un modelo predictivo ni una implementación de DevOps Guru/Cloud Monitoring Anomaly Detection. La configuración nativa y su evidencia siguen pendientes.

Observabilidad de red y seguridad

Envoy aporta visibilidad Este-Oeste de las dos dependencias HTTP: código, duración, destino upstream y contexto traceparent. Prometheus recoge las métricas de ambos proxies. Las solicitudes de entrada a service-a permiten observar el frente Norte-Sur local; la red de proveedor no está instrumentada en este ensayo.

Golden Signal / control	Estado y evidencia
Autenticación fallida	Ráfaga validada de 10 respuestas 401; 20 intentos totales. AuthFailures y security-validation.json.
Tráfico E-W	Métricas Envoy y access logs; alerta por tasa >100 req/s durante 10 s.
Tráfico N-S	Solicitudes y métricas del servicio de entrada; sin VPC Flow Logs cloud.
Vulnerabilidades conocidas	7 IDs únicos en Starlette; security-scan.json. No es un escaneo de imagen.
Control de caos	Token de operador y validación de rango; prueba de acceso sin token rechazada.
Seguridad cloud	Terraform declara Security Hub básico; SCC requiere configuración posterior.

Análisis de riesgo. Los hallazgos en dependencias no se ocultan ni se convierten en un cero por ausencia de integración. Deben actualizarse FastAPI/Starlette con versiones compatibles de OTel y volver a ejecutar todo el ensayo. El dashboard muestra una instantánea del escaneo, no un feed continuo de CVEs.

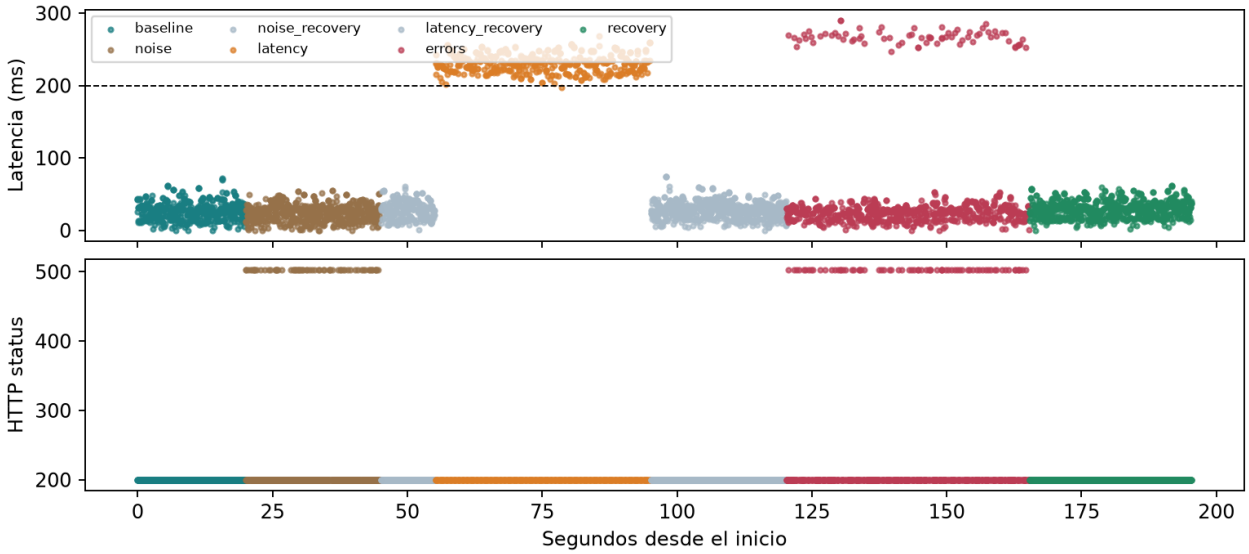
Brecha explícita. VPC Flow Logs GCP/AWS, SCC/Security Hub operativo, reglas sobre flujos cloud y hallazgos centralizados no fueron ejecutados. Las reglas L7 no reemplazan esos requisitos. La IaC valida estructura de proveedor, no permisos, capacidad disponible ni ejecución de recursos.

Actualización tecnológica: AWS documenta el fin de soporte de App Mesh el 30 de septiembre de 2026. Se propone acordar con la docente Istio/Envoy o ECS Service Connect como sustituto, conservando las métricas L7. La sustitución no se presume aprobada.

Experimento 1: latencia en service-b

Hipótesis. Al añadir 200 ms a service-b, el p99 supera el SLO y ServiceBLatency entra en firing en menos de dos minutos, conservando visibilidad de toda la cadena. Los errores no son necesarios para esta alerta de latencia.

Medición	Valor
Latencia inyectada	200 ms por petición al servicio B
MTTD observado	8.832 s
Solicitudes medidas	440
p99 extremo a extremo	257.26 ms
Tasa de errores	0.00%
SLI compuesto de la fase	0.23%



El cliente guarda cada petición y consulta el estado real de la alerta; la medición incorpora exportación, scrape, evaluación y observación. El registro events.json contiene inicio, firing y rollback. El TTL de 90 s y el bloque finally limitan la duración del fallo si el operador pierde el control.

Resultado: la degradación se detectó dentro del objetivo de 120 s. Los resultados se limitan a una ejecución en este sandbox; la capacidad, el jitter y la configuración cloud pueden cambiar el tiempo de detección.

Experimento 2: errores en data-service

Hipótesis. Una probabilidad Bernoulli de fallo del 10% en data-service, con 250 ms en las peticiones fallidas, activa simultáneamente la tasa de error anómala y p99 fuera de SLO. Esto permite probar exactamente la regla de correlación solicitada.

Medición	Valor
Probabilidad configurada	10%; semilla 7341 por proceso
MTTD observado	7.429 s
Solicitudes / errores medidos	764 / 73
Tasa de error observada	9.55%
p99 extremo a extremo	279.79 ms
Recuperación: errores	0

Ejemplo de alerta enriquecida conservada:

Servicio: data-service
trace_id: b9059bf4eba2a74e0915cec85bd15585
error_rate: 0.1333
p99: 0.2619 s
Acción: revertir fallo y revisar span de dependencia/consulta SQL.

La tasa observada puede diferir del 10% por el tamaño de muestra y porque el detector genera probes adicionales. El CSV documenta el tráfico cliente; las ventanas AIOps documentan sus probes. Son poblaciones relacionadas, pero no deben mezclarse para recalcular porcentajes.

Recuperación verificada. Se retira el fallo y se ejecutan 30 s de tráfico de extremo a extremo. verify.py comprueba ausencia de errores en esa fase y preserva una traza exitosa con PostgreSQL. La alerta de latencia de service-b puede activarse también durante errores lentos aguas abajo; esa propagación es esperada y no implica una nueva inyección en B.

Accionabilidad: el servicio responsable, trace_id y runbook reducen la búsqueda manual. No hay notificación externa ni automatización de remediación cloud; firing es el estado del motor de alertas, no una confirmación de recepción por un humano.

SLO, error budget e interpretación

Se define un SLO experimental del 99%: una solicitud buena responde sin 5xx/errores de conexión y tarda como máximo 200 ms. SLI = buenas/N. Presupuesto = 1% de N. Consumo de presupuesto = malas/(0.01 × N). Se reporta por fase, sin extrapolar a un mes de operación.

Fase	N	SLI	p99 ms	Consumo budget
baseline	512	100.0%	61.2	0.0x
noise	648	90.3%	50.9	9.7x
noise_recovery	252	100.0%	55.4	0.0x
latency	440	0.2%	257.3	99.8x
latency_recovery	632	100.0%	58.4	0.0x
errors	764	90.4%	279.8	9.6x
recovery	760	100.0%	56.8	0.0x

¿Se degradó el SLO? Se responde comparando cada SLI medido con el 99% objetivo. La latencia deliberada puede consumir el presupuesto aunque las respuestas sean exitosas. El ensayo de errores consume presupuesto por disponibilidad y por tiempo; una misma solicitud mala se contabiliza una sola vez.

¿Se consumió el error budget? El múltiplo de la tabla muestra la relación con el presupuesto de esa fase. Un valor mayor que 1 significa agotamiento. En una ventana corta el presupuesto puede ser fraccionario; por eso se utiliza una razón y no se redondea el número permitido de fallos a un entero.

¿La alerta fue accionable? Sí en el alcance local: el motor detectó la condición, conservó trace_id y permitió revertir el estímulo. La prueba no demuestra disponibilidad mensual, MTTR de un equipo real, resiliencia multirregión ni tolerancia a pérdida de la base gestionada.

Limitaciones: una repetición por fallo, baseline breve, p99 de ventanas pequeñas, carga local y detector de probes. El objetivo <120 s se verifica para cada observación; no se afirma una media estadística de MTTD. Para producción se requieren repeticiones, intervalos de confianza y presupuesto de 28/30 días.

Madurez en ocho dominios

Autoevaluación operacional de ocho dominios alineados con los temas del enunciado. La actividad no adjunta una taxonomía detallada del Blueprint; esta correspondencia se declara explícitamente. Escala: 1 inicial, 2 repetible local, 3 gestionado con SLO, 4 cuantificado en producción y 5 optimización continua. La puntuación no es calificación docente.

Dominio	Nivel	Justificación y brecha
Métricas y SRE	2	SLI y presupuesto calculados; sin ventana productiva.
Logging	2	OTLP/Loki correlacionado; falta gobierno y retención cloud.
Tracing y OTel	2	Tres servicios y DB; falta prueba multicloud y sampling.
AIOps	2	Baseline y AND medidos; falta proveedor nativo y evaluación etiquetada.
Network Observability	2	Envoy L7; sin flujos VPC ni mTLS gestionado.
Security Observability	2	401, alerta y escaneo; hallazgos pendientes, sin SCC/Hub activo.
DataOps	2	Esquema y calidad básica; sin frescura ni linaje continuo.
Resiliencia y operación	2	Dos fallos y rollback reproducibles; sin ejercicio grupal cloud.

Lectura de la madurez. El promedio es 2.0/5 en esta escala. Tener código y herramientas no equivale a capacidad productiva gestionada. Ningún dominio recibe un nivel superior por infraestructura aún no aplicada. La prioridad es cerrar las dependencias cloud y de seguridad antes de sofisticar el detector.

La evidencia es auditable: verification.json, resultados, logs, trazas y tablas conservan el estado observado. Los artefactos crudos permiten refutar o reproducir las conclusiones; las capturas complementan esa evidencia, pero no sustituyen los datos.

Roadmap, reproducción y referencias

Plazo / responsable	Entrega medible para alcanzar nivel 3
Mes 1 / plataforma	Cuentas sandbox y presupuesto; aplicar IaC; Cloud SQL/RDS privados; OTel en ambas clouds; evidencias verificadas.
Mes 2 / observabilidad y seguridad	Resolver hallazgos; malla compatible acordada; Flow Logs, SCC/Hub y alertas nativas; dataset etiquetado de incidentes.
Mes 3 / SRE y datos	SLO 30 días y burn rate; 5 repeticiones por caos; medir precisión/recall, MTTD y MTTR; frescura/linaje; sustentación grupal.

Reproducción. En la carpeta actividad 3: generar .env con scripts/init_env.py; iniciar docker compose up -d --build; esperar baseline; ejecutar scripts/experiment.py y scripts/verify.py. Consultar README.md y docs/RUNBOOK.md. Los secretos no forman parte de Git. CLOUD_HANDOFF.md describe las brechas de despliegue. Video_Demostracion_Local.mp4 registra una captura real de navegador, sin audio, y no una sustentación multicloud.

Referencias técnicas oficiales y fuente de evaluación

Universidad de La Sabana. Actividad 3.3 y rúbrica (consulta autenticada, 18/10/2026).
https://unisabanae-learning.unisabana.edu.co/mod/assign/view.php?id=306421

OpenTelemetry. Collector configuration.
https://opentelemetry.io/docs/collector/configuration/

OpenTelemetry. Semantic conventions for SQL.
https://opentelemetry.io/docs/specs/semconv/db/sql/

Grafana. Ingesting logs using OTLP.
https://grafana.com/docs/loki/latest/send-data/otel/

AWS. App Mesh: aviso de fin de soporte.
https://docs.aws.amazon.com/app-mesh/latest/userguide/

AWS. Amazon DevOps Guru.
https://docs.aws.amazon.com/devops-guru/latest/userguide/welcome.html

Google Cloud. VPC Flow Logs.
https://cloud.google.com/vpc/docs/flow-logs

Google Cloud. Security Command Center.
https://cloud.google.com/security-command-center/docs/concepts-security-command-center-overview